

Gabby's Study Program - LSAT Prep Simulator

A standalone desktop application, built in Unity, that simulates the LSAT end-to-end - timed multi-section practice tests drawn from a custom question bank, with persistent score tracking, full answer review, and a searchable question library.

Type: Solo personal project (built ~2020, ~5 years ago) - Role: Sole designer & developer - Positioning: Software Engineering (primary) -> Application / Game-Engine Development -> C# / Unity (secondary)

How to use this brief (for a resume assistant): This is detailed source material on an early, complete personal project. Use it to write or tailor resume bullets for a given role - select the most relevant details, keep all claims accurate, and don't overstate (it is a finished, self-built desktop app from early in the developer's learning, not a commercial product).

What it does

- Recreates the structure of the **LSAT**: three distinct section types - **Logical Reasoning**, **Reading Comprehension**, and **Analytical Reasoning (Logic Games)** - plus a combined **full-length timed test**.
- **Timed practice sessions** that mirror real exam pacing (35-minute sections; a multi-section full test), with an on-screen countdown, a visual timer bar, and a limited **pause** allowance.
- Builds each test by **randomly assembling questions** from a custom-authored question bank, so repeat attempts differ.
- **Tracks progress over time** - saves every completed attempt to disk, keeps a per-category history, and surfaces the user's **best score** in each section on the home screen.
- **Full post-test review**: correct/incorrect counts, a percentage score, and a question-by-question breakdown showing the correct answer and a source reference (book / page / question ID).
- **Searchable question library** for looking up and reviewing any individual question by category.

Architecture & how it works

- **Central state / page manager**. A single controller (MainControl) drives the whole app as a set of toggled UI screens - home, each section, full test, results, review, and search - all within one Unity scene.
- **Typed data model**. Questions are plain serializable C# classes: a Question (prompt, answer choices, source IDs, mark-for-review flag), an Answer (text, is-correct, is-selected), and MultiStepQuestions for passage-based items where one stimulus drives several sub-questions. Questions are grouped into named QuestionCategory sets.
- **Test builder**. A dedicated component (PrepTest) holds the question bank and assembles a randomized test on demand - selecting the right pool, shuffling, setting the section's timer, and tagging the test type.
- **Live test runner**. Separate runners for single questions (QuestionTemplate) and passage-based blocks (MultiStepQuestionControl) handle the countdown timer, answer capture, an answered-progress indicator, mark-for-review, and end-of-test logic.
- **Navigation grid**. A question-index grid lets the user jump to any question and see at a glance which are answered, unanswered, or flagged for review.
- **Persistence layer**. A save manager (SavedDataKeeper) serializes all results to an on-disk file (binary serialization), maintaining a full attempt history per category and tracking best scores across sessions.
- **Results & review**. The results screen (ResultPage) scores the attempt, computes the percentage, and renders a reviewable list of every question with its correct answer and source reference.

Feature decomposition

- **Section modes**. Each LSAT section is its own configured test type with its own question pool and timing; a full-test mode stitches the sections into a single long-form exam and produces a combined score.
- **Single vs. passage-based questions**. The model and runners support both standalone questions and multi-step blocks (a shared passage with multiple dependent sub-questions), matching real Reading Comprehension and Logic Games formats.
- **Exam-realistic controls**. Countdown timer with a visual bar, mark-for-review, a jump-anywhere question grid, an answered-count progress indicator, and a capped pause timer - the affordances of a real proctored test.
- **Source-referenced answers**. Every question carries a book / page / question ID so review traces back to the original study source.

Data & results lifecycle

- **Take** a randomized, timed test in any section or as a full-length exam.
- **Score & review** immediately - correct/incorrect, percentage, and a per-question walkthrough.
- **Persist** the attempt to disk with its date, time remaining, score, and full question set.
- **Track** a growing per-category history and an automatically updated best score per section.
- **Look up** any individual question later through the search library.

Key technical accomplishments

- Designed and shipped a **complete, self-contained desktop application** in Unity/C# - from data model and UI to scoring and persistence - as a sole developer.
- Built a **state-machine-style UI architecture** coordinating many screens and test modes through a single controller within one scene.
- Implemented a **data-driven test engine** that randomly composes timed tests from a typed question bank and supports both single and passage-based question formats.
- Engineered a **persistence system** (on-disk serialization) for cross-session score history and best-score tracking, including a results model that stores full attempts for later review.
- Created an **exam-faithful UX** - timer, pause, mark-for-review, navigable question grid, and detailed answer review with source references.

Tech stack

Language / Engine: C#, Unity 2019.2 **Platform:** Windows standalone (.exe) desktop build **UI:** Unity UI (uGUI) - canvas-driven screens, dynamic instantiation **Persistence:** .NET binary serialization to disk **Tooling:** Git, Visual Studio

Skills demonstrated

C# - Unity - object-oriented design - application architecture - UI state management - data-driven design - data modeling - serialization & persistence - save/load systems - randomized content generation - timed-event / game-loop logic - UI/UX implementation - desktop application development - solo end-to-end delivery - Git

Scope & honest status

- **Status:** a finished, working desktop app that was actually built and used - a complete project, not a demo.
- **Context:** created early in the developer's learning (~5 years ago). The codebase shows that origin - e.g. one placeholder class left empty and some debug scaffolding - and content (the question bank) was authored by hand.
- **Best framed as:** an early, fully-completed application that demonstrates real C#/Unity fundamentals - architecture, data modeling, persistence, and UX - and the ability to take a personal project from idea to a working, distributable build solo.

Ready-to-use resume bullets

- Designed and built a complete **standalone LSAT prep desktop application** (Unity, C#) from scratch as sole developer - data model, UI, test engine, scoring, and persistence.
- Implemented a **data-driven test engine** that randomly assembles timed, multi-section practice exams from a typed question bank, supporting both single and passage-based (multi-sub-question) formats.
- Architected a **single-controller, multi-screen UI** coordinating section modes, full-length exams, results, review, and search within one Unity scene.
- Engineered a **save/load persistence layer** (on-disk serialization) for cross-session attempt history and automatic best-score tracking per section.
- Delivered an **exam-realistic UX** - countdown timer, pause allowance, mark-for-review, a jump-anywhere question grid, and question-by-question answer review with source references.